TROC16_4sz operator list	**James C Brakefield © 2025**	16, 24 & 32-bit instructons; four data sizes up to register size		
		"n" immediate field is a value of -4..7, first 4-bits of 12-bit immediate, or length of postfixed immediate in bytes		
data sizes: byte(8),half(16),3/4ths word(24),word(32)		last four registers are residue, frame pointer, stack pointer & PC		
Y scale factor is data size times (1 to 4)		residue as base register & PC as index register read as zero		
Type of operation determined by types of the operands, so same op-code for unsigned, signed, float or float2; mismatched operand types may cause a trap				
each register has a 2-bit type code: unsigned, signed, float & float2; # inuse tag bits varies		32-bit instructions: residue update optional		
first six bits (5..0) of op-code in LSB, other two bits in third byte (bits 17..16); S and D fields not relocated from 16-bit instructions				

format name	**instruction layout**	**usage**		**address arithmetic**
fmt16di	nnnnn ddddd 1xxx11	immediate instruction D op I ->D: SHFT,EXTCT,AOB,SOB, LDIU,LDIS,LDIF,LDIF2		
fmt16ds	sssss ddddd 0xxx01	basic 16-bit instruction S op D ->D ADD,SUB,MUL,DIV,CMP,AND,OR,XOR		
fmt16di	nnnnn ddddd 0xxx11	16-bit immediate instruction D op I ->D ADDI,SUBI,MULI,DIVI,CMPI,ANDI,ORI,XORI		
fmt16dn	nnnnn ddddd 111x11	load/store tagged to memory location N		D -> M(N<<2)?
fmt16dn	nnnnn ddddd 1xxx01	INN, OUTN, BSR, MOV, BRN, BRP, BRZ, BRNZ		PC + N

format name	**instruction encoding & neumonics**		**usage**		**arithmetic**
fmt16ds, fmt16di	000101 000111	ADD, ADDI	D sets the data type, mixed type traps		S + D -> D, N + D -> D
fmt16ds, fmt16di	001101 001111	SUB, SUBI	D sets the data type, mixed type traps		S - D -> D, N - D -> D
fmt16ds, fmt16di	010101 010111	MUL, MULI	D sets the data type, mixed type traps		S * D -> D, N * D -> D
fmt16ds, fmt16di	011101 011111	DIV, DIVI	D sets the data type, mixed type traps		D / S -> D, D / N -> D
fmt16ds, fmt16di	100101 100111	AND, ANDI	typeless, no change to D type		S and D -> D, N and D -> D
fmt16ds, fmt16di	101101 101111	OR, ORI	typeless, no change to D type		S or D -> D, N or D -> D
fmt16ds, fmt16di	110101 110111	XOR, XORI	typeless, no change to D type		S xor D -> D, N xor D -> D
fmt16ds, fmt16di	111101 111111	CMP, CMPI	residue register format TBD		D ? S -> Res, D ? N -> Res
fmt16dn	000001	INN	input from IO port N		port N -> D if |n| < 16
fmt16dn	001001	OUTN	output to IO port N		D -> port N if |n| < 16
fmt16dn	000001	LDN	M (N) => D		M(N) -> D if |n| > 15
fmt16dn	001001	STN	output to IO port N		D -> M(N) if |n| > 15
fmt16dn	100001	BRN	branch relative negative		PC + N -> PC if MSB = 0
fmt16dn	101001	BRP	branch relative positive		PC + N -> PC if MSB = 1
fmt16dn	110001	BRZ	branch relative zero		PC + N -> PC if D = 0
fmt16dn	111001	BRNZ	branch relative not zero		PC + N -> PC if D /= 0
fmt16di	000011	SHFTN	shift D per signed N		D << N -> D if |n| < 16
fmt16di	001011	FUNC	function n on D		fn(D) => D if |n| < 16
fmt16di	000011	INSRTN	insert bits from R into D per signed N		bits per N from D => R if |n| > 15
fmt16di	001011	EXCTN	extract bits using 12-bit N from D to R, typeless		extract bits 'N' from D => R if |n| > 15
fmt16di	100011	LDIU	load immediate unsigned		I -> D as unsigned
fmt16di	101011	LDIS	load immediate signed		I -> D as signed
fmt16di	110011	LDIF	load immediate float		I -> D as float
fmt16di	111011	LDIF2	load immediate float2		I -> D as float2

format name	**instruction layout**	**usage**		**address arithmetic**

fmt24drsi		rrrrr ixx sssss ddddd xxxx10	basic 24-bit instruction		
fmt24drs		rrrrr 0xx sssss ddddd xxxx10	R op S ->D		
fmt24dri		iiiii 1xx sssss ddddd xxxx10	immediate op S->D		
fmt24dbx		jjjjj 0xx bbbbb ddddd xxxx10	load/store base + scaled index		B + J * ds
fmt24dbn		nnnnn 1xx bbbbb ddddd xxxx10	load/store base + offset		B + N
fmt24dr		kkkkk zxx sssss ddddd xxxx10	op K -> D; 32 single operand instructions		
fmt24dn		nnnnn nxx nnnnn ddddd xxxx10	eleven bit signed offset or displacement		PC + N
format name	**instruction layout**		**usage**		**address arithmetic**

fmt32drsc	ttttt zzu rrrrr ixx sssss ddddd xxxx00	basic 32-bit instruction		
fmt8	xx000000	trap, breakpoint & align inst		trap on all zeros
fmt32dbjt	ttttt yyu jjjjj 0xx bbbbb ddddd xxxx00	load from base + index * scaling + offset		B + J * Y * ds + T
fmt32dbjn	nnnnn yyu jjjjj 1xx bbbbb ddddd xxxx00	load from base + index * scaling + offset		B + J * Y * ds + N
fmt32dbjt	ttttt yyu jjjjj 0xx bbbbb ddddd xxxx00	store to base + index * scaling + offset		B + J * Y * ds + T
fmt32dbjn	nnnnn yyu jjjjj 1xx bbbbb ddddd xxxx00	store to base + scaled index + offset		B + J * Y * ds + N
fmt32mbjt	ttttt yy1 jjjjj 0xx bbbbb mmmmm xxxx00	store immediate to base + index * scaling + offset		B + J * Y * ds + T
fmt32mbjn	nnnnn yy1 jjjjj 1xx bbbbb mmmmm xxxx00	store immediate to base + index * scaling + offset		B + J * Y * ds + N
fmt32ds	ccccc kku kkkkk zxx sssss ddddd xxxx00	op S -> D; 128 single operand instructions		function K of S ->D
fmt32drst	ttttt zzu rrrrr ixx sssss ddddd xxxx00	op(R, S, T) -> D; triple operand instructions		
fmt32dn	nnnnn nnn nnnnn nxx nnnnn ddddd xxxx00	nineteen bit signed offset or immediate		PC + N

	40-bit instructions within 24-bit instruction code space, 9 op-code bits			
fmt40dsrct	ttttt xxx ccccc uxx rrrrr ixx sssss ddddd 11xx10	40-bit instruction with three source registers & predication		
fmt40dsret	ttttt xxx eeeee uxx rrrrr ixx sssss ddddd 11xx10	40-bit instruction with three source registers, 2 destination registers		
	48-bit instructions within 32-bit instruction code space, 12 op-code bits			
fmt48dsrcet	ttttt eeeee xxxxxx ccccc uxx rrrrr ixx sssss ddddd 11xx00		basic 48-bit instruction with 3 source, 2 destination & pre	
fmt48dsrwet	ttttt eeeee xxxxxx wwwww uxx rrrrr ixx sssss ddddd 11xx00		basic 48-bit instruction with 4 source, 2 destination	

24-bit formats	**32-bit formats**	**neumonics**	**short description (43 of 64 allocated, 16 reserved for 40 & 48 bit instructions)**	
				# op codes
	Prunned instructions in grey, e.g. need to use 40-bit instruction with its additional op-code bits			
fmt24dbj, fmt24dbn	fmt32dbjs, fmt32dbjn	LDU8, LDU16, LDU24, LDU32	load unsigned byte/half/word from memory	
fmt24dbj, fmt24dbn	fmt32dbjs, fmt32dbjn	LD8, LD16, LD24, LD32	load signed byte/half/word from memory	
fmt24dbj, fmt24dbn	fmt32dbjs, fmt32dbjn	FLD8, FLD16, FLD24, FLD32	floating-point load	
fmt24dbj, fmt24dbn	fmt32dbjs, fmt32dbjn	F2LD8, F2LD16, F2LD24, F2LD32	Float2 load	
	fmt32dbjs, fmt32dbjn	LEA8, LEA16, LEA24, LEA32	load effective address	
fmt24dbj, fmt24dbn	fmt32dbjs, fmt32dbjn	ST8, ST16, ST24, ST32	store byte/half/word to memory	32-bit version supports store immediat
fmt24drs, fmt24drn	fmt32drst, fmt32drsn	ADD, ADDI	R + S -> D	
fmt24drs, fmt24drn	fmt32drst, fmt32drsn	SUB, SUBI	R - S -> D	
fmt24drs, fmt24drn	fmt32drst, fmt32drsn	MUL, MULI	R * S -> D	
fmt24drs, fmt24drn	fmt32drst, fmt32drsn	MAC, MACI	multiply-add	24-bit: R * S +D -> D?
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	DIV, DIVI	S / R -> D	

fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	AND, ANDI	R and S -> D	type code ignored	
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	OR, ORI	R or S -> D	type code ignored	
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	XOR, XORI	R xor S -> D	type code ignored	
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	CMP, CMPI	R : S -> D, result to D register	My 66000 CCR format plus even/odd bits	
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	FATAN2PI	arc-tangent in rotations for two operands		
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	FPOW, FPOWI	R raised to the S power		
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	INSRT, INSRTI	bit field insert	type code ignored	starting bit and length is 12-bit contiguc
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	EXTRCT, EXTRCTI	extract bit field	type code ignored	starting bit and length is 12-bit contiguc
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	ROL, ROLI	rotate left	type code ignored? Use extract with full bit count?	
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	SHR, SHRI	shift right		single shift inst via type code?
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	ASR, ASRI	arithmetic shift right		single shift inst via type code?
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	SHL, SHLI	shift left		single shift inst via type code?
fmt24cbj, fmt24cbn	fmt32cbjs, fmt32cbjn	JMPcc, CALLcc	conditional jump/call	D is the condition, JMP-never mapped to CALL	
fmt24cn	fmt32cn	BRcc, BSRcc	conditional relative branch/call with 1	D is the condition, BR-never mapped to BSR	
fmt24drs, fmt24drn	TBD	BBS, BBC	relative branch on bit set/clear	N: branch delta, R: source, D: bit number; requires 4 op-c	
fmt24crn	fmt22crbj, fmt22crbn	BRccRC, JMPccRC	conditional relative branch/jump on register contents		
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	MAX, MAXI	maximum		
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	MIN, MINI	minimum		
	fmt32drsc, fmt32drnc	MEDIAN	median		
fmt24drs, fmt24drn	fmt32drsc, fmt32drnc	EADD, EADDI	add to exponent		via type tags use shift inst?
fmt24dn	fmt32dn	VECT	identify loop vector registers		
fmt24drs, fmt24drn	TBD	LOOP, LOOPI	vector loop instruction		
fmt24dr	fmt32dre	FUNC	single operand instructions	operand can be negated/inverted	
fmt24dr	fmt32dre	P3OP	16X 24-bit instructions with additional operation bits, no load/store instructions		
		CASE, CASEI	select PC or PC displacement from table		
	fmt32dsrn	CMPBR, CMPBRI	compare and branch on condition		
fmt24n		OPCDLOC	For first instruction in cache or DRAM line, indicates all op-code locations		
	fmt8	TRAP	fatal error		
	fmt8	BKPT	breakpoint		
	fmt8	ALIGNW, ALIGNC	32-bit alingment		also for cache alignment
		MISC Probably useful instructions			using zzu bits as opcode bits
fmt24drs, fmt24drn	TBD	MMOV	memory to memory byte string move	My 66000 instruction	
	TBD	PCND	create predicate shadow on condition	My 66000 instruction	
	TBD	PCB1	create predicate shadow on bit	My 66000 instruction	
	TBD	STM	multi-register save to memory	My 66000 instruction	
	TBD	LDM	multi-register load from memory	My 66000 instruction	
	TBD	CASE	indexed jump with max limit	My 66000 instruc	should PC be implied?
	TBD	MUX	select one of two inputs		
	TBD	MERGE	select bits from one of two inputs		pared with XOR
					Totals

Single Operand Instructions (29 allocated of 32 in fmt24, or 86 of 128 in fmt32), operand negate/invert supported					
	Prunned 24-bit instructions in grey, e.g. need to use 32-bit instruction with its additional op-code bits				
fmt24dr	fmt32dr	IN	read input port	R is the port number	
fmt24dr	fmt32dr	OUT	write to output port	R is the port number	
fmt24dr	fmt32dr	MOV, MOVI	move register	macro op	
fmt24dr	fmt32dr	NOT	Boolean complement	macro op	
fmt24dr	fmt32dr	NEG	Negate	macro op	
fmt24dr	fmt32dr	INC, DEC	Increment, Decrement	macro op	
fmt24dr	fmt32dr	ABS, NABS	Absolute value	macro op	
fmt24dr	fmt32dr	LDZCNT, LD1CNT, TRZCNT, TR1CNT	Leading and trailing zero/ones count	ranges from zero to register size	
fmt24dr	fmt32dr	POPCNT	Count of one/zero bits	ranges from zero to register size	
fmt24dr	fmt32dr	SINPI, COSPI, TANPI	Trig functions in half rotations		
fmt24dr	fmt32dr	ASINPI, ACOSPI, ATANPI	Trig functions in half rotations		
fmt24dr	fmt32dr	INT, FLT	Integer from/to float		
	fmt32dr	CVT (48)	to & from: flt, flt2, signed, unsigned	My 66000 instruc	16-bit inst, conversions & coercions
fmt24dr	fmt32dr	EXPON	extract exponent		
fmt24dr	fmt32dr	FRACT	extract fraction		
fmt24dr	fmt32dr	SQRT	square root		
fmt24dr	fmt32dr	FSQRT	floating-point square root		
fmt24dr	fmt32dr	FRSQRT	floating-point reciprical square root		
fmt24dr	fmt32dr	FRCP	reciprical		
fmt24dr	fmt32dr	LN2P1	log to base 2 plus one		
fmt24dr	fmt32dr	LN2	log to base 2		
	fmt32dr	LNP1	natural log plus one		
	fmt32dr	LN	natural log		
	fmt32dr	LOGP1	base 10 logarithm plus one		
	fmt32dr	LOG	base 10 logarithm		
fmt24dr	fmt32dr	EXP2M1	base 2 exponentation minus one		
fmt24dr	fmt32dr	EXP2	base 2 exponentation		
	fmt32dr	EXPM1	base E exponentation minus one		
	fmt32dr	EXP	base E exponentation		
	fmt32dr	EXP10M1	base 10 exponentation minus one		
	fmt32dr	EXP10	base 10 exponentation		
	fmt32dr	SIN	trig functions in radians		
	fmt32dr	COS	trig functions in radians		
	fmt32dr	TAN	trig functions in radians		
	fmt32dr	ASIN	trig functions in radians		
	fmt32dr	ACOS	trig functions in radians		
	fmt32dr	ATAN	trig functions in radians		
fmt24dr	fmt32dr	RND	round using current mode		

fmt24dr	fmt32dr	RNDTEV	round to nearest, ties to even		
fmt24dr	fmt32dr	RNDTOD	inexact round to nearest odd		
fmt24dr	fmt32dr	RNDTZ	round toward zero		
fmt24dr	fmt32dr	RNDFZ	round away from zero		
fmt24dr	fmt32dr	CEIL	ceiling		
fmt24dr	fmt32dr	FLOOR	floor		
fmt24dr		USGND	unsigned coercion		16-bit instruction
fmt24dr		SGND	signer coercion		16-bit instruction
fmt24dr		FLT	foat coercion		16-bit instruction
fmt24dr		FLT2	float2 coercion		16-bit instruction
					Totals